

INTERNAL TECHNICAL DOCUMENTATION

Ghrab VOC Vulnerability Operations Center

Architecture, agent design, persistence layer, and technology-stack review of the AI-augmented, multi-tenant VOC platform — prepared for the Vulnerability Management team.

Prepared: 16 July 2026

Repository: `voc` · branch `rework` · commit `0767ee6` "Added persistence, dbs, lots of things"

Scope: deterministic engine, 7-agent AI layer, SQLite persistence, multi-tenant dataset registry, REST/SSE API, React SPA, deployment

Audience: VM team leadership + engineers who will maintain/extend the platform

Table of Contents

1	Executive Summary
2	What This Platform Is (and Isn't)
3	Technology Stack at a Glance
4	System Architecture & Data Flow
5	The Deterministic Engine
5.1	Ghrab Risk Score (GRS)
5.2	Capability Classification
5.3	Autonomous Attack-Path Discovery
5.4	CMDB Grounding (No Pre-Solved Paths)
6	Persistence, Multi-Tenancy & Run Lifecycle
6.1	SQLite/SQLAlchemy Persistence
6.2	Runtime-Switchable Enterprises
6.3	Duplicate Detection, Reuse & the Kill Switch
6.4	The Held-Out Verification Oracle
7	The AI Agent Layer
7.1	Orchestrator & Pipeline
7.2	Multi-Provider Architecture (litellm)
7.3	The Seven Agents, In Detail
7.4	Anti-Hallucination / Grounding Design
8	API Surface (REST & SSE)
9	Frontend Application
10	Deployment & Operations

11 Testing & Quality Assurance — Current State

12 Recommended Improvements to the Tech Stack

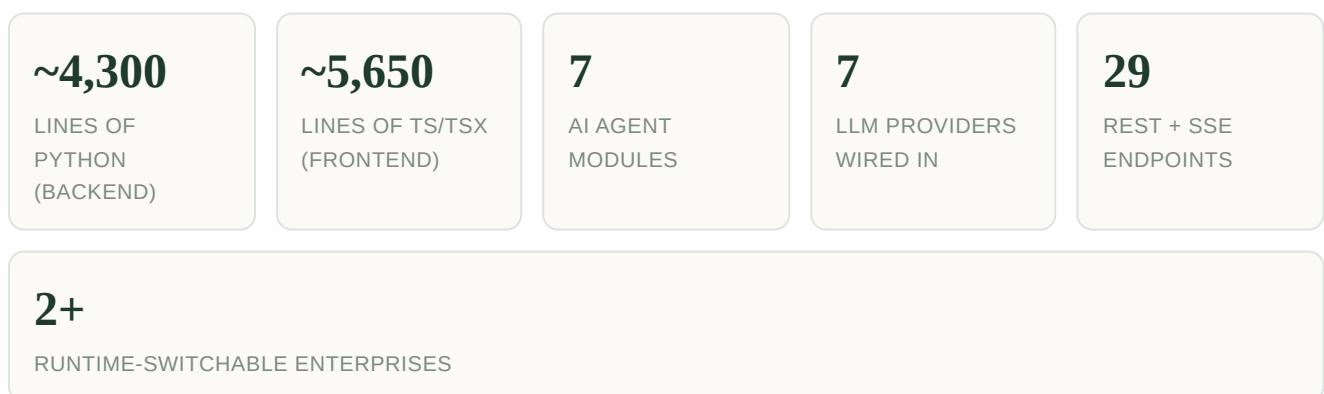
13 Glossary

1 • Executive Summary

The Ghrab VOC is a self-contained, AI-augmented Vulnerability Operations Center. It ingests a raw vulnerability scanner export (Qualys-style CSV) and an enterprise architecture document (a Markdown CMDB), and produces several layers of value on top of the raw data:

1. **A deterministic, explainable risk score** (the Ghrab Risk Score, or GRS) for every finding — combining CVSS, exploit-probability (EPSS), known-exploitation status (KEV), asset criticality, and blast-radius, gated by network exposure. This requires no AI and no API key.
2. **An autonomous attack-path discovery engine** that builds a reachability graph over the real asset inventory and independently derives which findings chain together into an actual internet-to-crown-jewel attack path — computed from first principles, never read from a pre-authored script.
3. **A seven-agent AI reasoning layer**, built on a vendor-neutral LLM abstraction (litellm) supporting seven model providers with automatic fallback, that validates, narrates, cross-references, and — as of the platform's newest agent — independently *detects* attack paths from raw grounding alone, with no candidate list and no answer key.
4. **A persistence and multi-tenancy layer**, added in the most recent development cycle: every analysis run is now stored in a SQLite database (swappable to Postgres via one environment variable), the platform can scan any number of "enterprises" dropped into its data directory and switch between them at runtime, byte-identical re-analysis is detected and offered for reuse instead of silently re-spending LLM tokens, and an operator can cancel a running pipeline mid-flight.

The system is delivered as two services — a FastAPI backend and a React/TypeScript single-page application — connected by a JSON + Server-Sent-Events API, and ships with Docker Compose for one-command deployment.



This document is a complete, from-scratch technical map of the platform as it exists today — including the persistence layer, multi-tenant dataset registry, and the newly redesigned "held-out oracle" verification strategy that did not exist in earlier iterations of this system. Section 12 closes with a candid assessment of where the technology stack still has gaps before this could move from a lab/demo posture toward a production-facing tool.

2 · What This Platform Is (and Isn't)

The core idea

Most vulnerability dashboards sort a flat list of CVEs by CVSS score. That fails a very common real-world scenario: a CVSS 10 sitting on an isolated, admin-only internal VM is often *less* urgent than a CVSS 7 sitting on an internet-facing box that chains into a database server holding customer records. The Ghrab VOC's central design bet is that **risk is a function of the finding plus where it sits in the network plus what it connects to** — not the finding alone.

It answers this in three independent, verifiable ways:

- **Quantitatively** — the GRS formula (Section 5.1) folds reachability and blast-radius directly into the score, so the ranked findings list already reflects context, before any AI is involved.
- **Structurally** — the deterministic attack-path engine (Section 5.3) builds an actual graph of the network and proves, hop by hop, which findings compose into a real attacker path from the internet to a crown-jewel asset.
- **Agentically** — a dedicated AI agent (Section 7.3, "Analyst Detection Agent") is handed *only* the raw asset/ownership/reachability/credential grounding — no candidate chains, no answer key — and reasons out attack paths the way a human analyst would, with every hop it proposes checked against real hosts and findings before it's shown to an operator.

Key architectural principle: the platform never lets the AI compute or override risk. GRS is plain Python arithmetic; the reachability graph is deterministic graph traversal (NetworkX) over data parsed from the CMDb. The AI agents narrate, rank, cross-reference, and independently re-derive — but every AI output is post-parse validated against deterministic ground truth before it reaches the UI (Section 7.4). This is what lets the dashboard run fully, with real risk scores and real attack paths, with **zero API keys configured**.

What's new since the last major revision

The most recent development pass ("Added persistence, dbs, lots of things") changed the platform's shape substantially:

- Analysis results now **persist to a real database** instead of a single JSON cache file — every run is a row in an `analysis_run` table, queryable by dataset and content fingerprint.
- The platform is no longer hardcoded to one organization — it **auto-discovers every scannable "enterprise"** in its data directory and lets an operator switch between them live, from the UI.
- The "answer key" architecture was redesigned: documented attack paths are no longer parsed out of the ingested architecture document at all — they live in a completely separate, held-out **verification oracle** that nothing in the detection pipeline can see.
- A new, more ambitious **Analyst Detection Agent** was added — unlike every other agent, it is given zero pre-computed candidate paths and must reason out attack chains entirely on its own.

What it isn't (yet)

- It is **not connected to a live scanner** — it ingests a static CSV export, not a Qualys/Tenable/Rapid7 API.
- It is **not multi-user or authenticated** — there is no login, no per-analyst workspace, no RBAC (Section 12).
- It currently ships with **two synthetic lab datasets** (a fictional financial-services org and a fictional healthcare org), not live organizational data — though the platform will now happily scan a third, fourth, or fifth if one is dropped into its data directory.

3 · Technology Stack at a Glance

Backend

LAYER	TECHNOLOGY	PURPOSE
Web framework	FastAPI 0.115	REST + Server-Sent-Events API, auto-generated OpenAPI docs at /docs
ASGI server	uvicorn	Dev/prod server, hosts the FastAPI app
Data handling	pandas	Loads and enriches the vulnerability CSV into a DataFrame — the working data model throughout the backend
Graph engine	networkx	Directed reachability graph construction + shortest-simple-paths enumeration for attack-path discovery
LLM abstraction	litellm ≥1.51	Single call interface (litellm.completion) across 7 model providers — no vendor SDK is called directly anywhere in the codebase
Persistence NEW	SQLAlchemy ≥2.0 + SQLite	ORM + engine for the analysis_run table; DATABASE_URL env var swaps to Postgres with no code change
Validation	pydantic	Request/response body models for the REST API
Config	python-dotenv	Loads provider API keys from a root .env file
Runtime	Python 3.12	

Frontend

LAYER	TECHNOLOGY	PURPOSE
Framework	React 18 + TypeScript 5	SPA, function components + hooks throughout
Build tool	Vite 5	Dev server with API proxy, production bundler
Routing	react-router-dom 6	9 top-level routes under a shared Layout , with animated route transitions (framer-motion)
Server state	@tanstack/react-query 5	Data fetching/caching for every REST call
Tables	@tanstack/react-table 8	Sortable findings table
Charts	recharts 2	Band-distribution bars, CVSS-vs-GRS scatter, team risk bars
Graph visualization	cytoscape 3 + cytoscape-fcose + cytoscape-navigator	Interactive, canvas-rendered attack-surface graph with 3 layout engines, replay, minimap (Section 9.1)
Styling	Tailwind CSS 3	Utility-first styling, light/dark theme via CSS custom properties
Motion	framer-motion	Route transitions, toast stack, micro-interactions

Icons	lucide-react	
-------	--------------	--

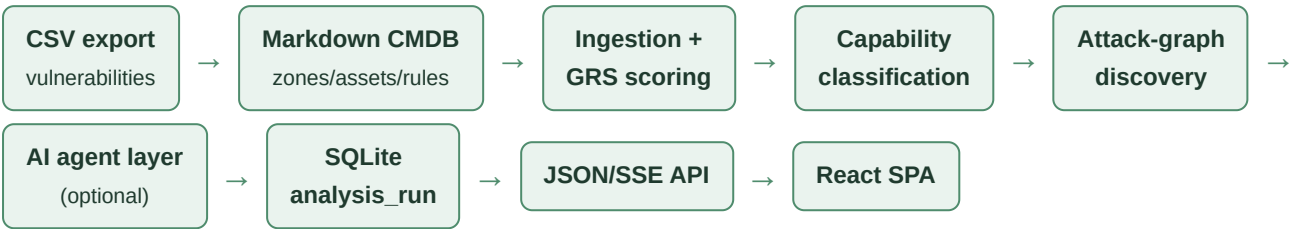
Infrastructure & delivery

CONCERN	APPROACH
Containerization	Two Dockerfiles (Python slim + Node build → nginx static serve); orchestrated with <code>docker-compose.yml</code>
Reverse proxy / TLS	nginx in the frontend container terminates TLS, redirects 80 → 443, serves the SPA, and reverse-proxies <code>/api/*</code> to the backend with buffering disabled (required for SSE to flush)
Local dev	<code>dev.sh</code> starts both services together; Vite's dev-server proxy handles <code>/api</code> without nginx
Persistence NEW	SQLite file at <code>backend/data/voc.db</code> , created automatically on first startup by <code>db/session.py::init_db()</code> — no manual migration step. The Docker Compose volume mount now covers the whole <code>backend/data</code> directory (previously just the cache subfolder) so the database and oracle files survive container restarts.
Secrets	LLM API keys via <code>.env</code> (server-wide) or pasted at runtime into an in-memory settings dict from the Settings screen — session-state wins over env var if both are set

Notably still absent from the stack: no authentication/authorization layer, no message queue, no observability/APM tooling, no CI pipeline, and no live threat-intel integration (EPSS/KEV are hardcoded lookup tables, not API calls). These are addressed as improvement items in Section 12.

4 • System Architecture & Data Flow

The platform is two services talking over HTTP/SSE. The backend now owns a small SQLite database in addition to its in-memory state — a meaningful shift from the prior JSON-cache-only design.



This is the sequence run by `agents/orchestrator.py`, now deliberately split into two functions:

- `compute_deterministic()` — the instant, no-API-key-needed half (CSV → GRS → capability → graph). Safe to call on every request; never spends a token, never touches the database.
- `run_ai_layer()` — the token-spending half (all AI agents). Only ever invoked from `api/state.py::run_new_analysis()`, which is reached exclusively via the explicit `POST /api/analyze` action — never as a side effect of a plain page-load GET.

Request/response model

There is a single active `AnalysisResult` held in server memory (`api/state.py`), but it is no longer the only copy of the truth: every completed run — deterministic-only or AI-enriched — is persisted as an `analysis_run` row. On a cold backend restart, `get_analysis()` recomputes the cheap deterministic layer immediately and **rehydrates the AI-enriched layer from the last persisted run** for the active dataset, rather than coming up blank. `POST /api/analyze` always computes fresh and persists a new run; a second connecting client (or a reloaded tab) attaches to the same in-flight run instead of starting a redundant one, exactly as before.

Implication: this is still a single-tenant-at-a-time backend process — there is exactly one "active" dataset and one "active" in-memory analysis shared by every connected client. Multi-tenancy here means "switchable," not "concurrent": two operators cannot currently view two different enterprises simultaneously from the same backend instance. See Section 12.

Live signal channels (SSE)

STREAM	ENDPOINT	PURPOSE
Analysis progress	<code>POST /api/analyze</code>	Streams each pipeline stage as it completes, plus new <code>duplicate</code> and <code>cancelled</code> events (Section 6.3) — drives the progress overlay in the SPA
AI Analyst chat	<code>POST /api/chat</code>	Streams the Triage Agent's answer word-by-word for a live-typing feel

Agent activity log	GET <code>/api/logs/stream</code>	Tails every LLM call the platform makes in real time — which agent, which provider, success/error/fallback, latency, tokens, cost (Section 7.2)
--------------------	--------------------------------------	---

5 • The Deterministic Engine

Everything in this section runs in pure Python with no network calls and no LLM. It is the trust anchor of the whole platform — the AI layer explains and extends this output, it never computes or overrides it.

5.1 • Ghrab Risk Score (GRS)

Implemented in `backend/core/risk_engine.py`, following the methodology in `ghrab_risk_methodology.md` — unchanged by the recent redesign. It's a deliberate synthesis of five recognized industry approaches (Tenable VPR / attack-path "toxic combinations", FIRST.org EPSS, CISA SSVC's exposure-gating idea, the FAIR risk model's contact-frequency concept, and EU DORA's critical-function weighting) into one composite, auditable number:

```
GRS = min(100, ImpactScore × ExposureTier × CCF)

ImpactScore = 10 × (0.30·CVSS + 0.15·EPSS×10 + 0.15·KEV(0/10) + 0.20·ACW×10 + 0.20·TCM)
```

FACTOR	WEIGHT	SOURCE
CVSS Base Score	30%	As-scanned, from the CSV
EPSS (exploit probability)	15%	Hardcoded per-CVE lookup table (<code>EPSS_BY_CVE</code>) — see Section 12
KEV (known exploited, binary)	15%	Hardcoded CISA KEV membership set (<code>KEV_CVES</code>)
ACW — Asset Criticality Weight	20%	Hand-derived 0–1 score per finding from the asset's business-criticality tier
TCM — Toxic Combination Score	20%	Hand-derived 0–10 score reflecting the finding's position in a documented attack chain (terminus steps score highest) — this enrichment table is derived from the oracle's chains at authoring time, but the score itself ships as a static lookup, not a live computation

The result is **gated** — not additively adjusted — by an `ExposureTier` multiplier reflecting network reachability (Internet-facing 1.30 → Adjacent 1.15 → Internal 0.90 → Restricted 0.50 → Isolated 0.15), and by a Compensating Controls Factor (CCF, currently always 1.0 — no verified compensating controls modeled). This multiplicative gate produces the reordering effect the design is built around: an unreachable critical finding is suppressed toward the bottom of the range; a reachable moderate finding can be pushed into the danger zone.

ACTION BANDS (SLA)

BAND	GRS RANGE	SLA
IMMEDIATE	80–100	24–72h, emergency change

ACT	60–79	7 days
ATTEND	40–59	30 days
TRACK*	20–39	90 days / next patch cycle
TRACK	0–19	Monitor only

A separate overlay applies for findings flagged as touching an EU DORA "Critical or Important Function" (CIF): the remediation SLA is capped at 30 days regardless of the computed band. The QID/CVE-keyed enrichment tables in `risk_engine.py` hold entries for every lab dataset shipped (Ghrab and Velon); QID/CVE namespaces don't collide across labs, so the tables are simply unioned rather than switched on the active enterprise — `compute_grs()` has no notion of "which enterprise" it's scoring, only "which QID/CVE."

5.2 • Capability Classification

`backend/core/capability.py` is the first half of the discovery engine, unchanged in its core logic. Every finding is mapped — from its own CVSS vector, category, CVE, and free-text description/consequence, using compiled regex keyword signals — into what an attacker actually *gains* from it, in ATT&CK-style terms: `initial_access`, `rce_system / rce_user`, `credential_theft` (with a domain-admin-yielding sub-flag), `privilege_escalation / cloud_priv_esc`, `segmentation_break` (the edge-forming capability the graph engine keys on), and `impact`.

Constraint enforced by design: this classification is derived entirely from the finding's own content — it has no access whatsoever to the held-out oracle (Section 6.4). It never has, and the recent redesign made this structurally impossible rather than just a convention: the answer key isn't merely unused, it no longer exists anywhere in the ingested data at all.

5.3 • Autonomous Attack-Path Discovery

`backend/core/attack_graph.py` composes the per-finding capabilities into a directed reachability graph over the real asset inventory, then enumerates chains from attacker-reachable entry points to the organization's crown jewels.

GRAPH CONSTRUCTION (SIX EDGE TYPES)

- **Entry edges** — INTERNET → any host carrying an internet-reachable, unauthenticated finding.
- **Same-zone lateral movement** — hosts in the same VLAN can reach each other (flat L2 segment assumption).
- **Tier-aware app → db adjacency** — an app server reaches only its own database (matched by business-function token in the hostname), not the whole DB tier.
- **Segmentation-break edges** — a finding that opens a VLAN boundary adds a directed edge between the zones/hosts it connects, tagged with the enabling QID.
- **Credential-reuse edges** — shared local credentials link the specifically named hosts, symmetrically.

- **Domain-Admin edges** — a finding that yields Domain Admin adds an edge to every AD-joined host in the domain-trust VLANs.

PATH ENUMERATION & SCORING

For each host flagged a **crown jewel**, the engine runs NetworkX's `shortest_simple_paths` from INTERNET, prunes paths through any "empty" intermediate host with no finding and no specific enabling edge, then scores survivors:

```
score = target_value × 45 + max_GRS × 0.45 + blast_radius × 8 - length_penalty
```

Paths are deduplicated by (entry, target) keeping the best-scoring representative, then diversified so every distinct crown-jewel target is represented before filling out the ranked list by score alone.

Changed in this cycle: `discover_paths()` previously always capped its output at a fixed top-14. The cap (`top_n`) is now `None` by default — the function surfaces every distinct entry → crown-jewel path the reachability graph supports, since "how many attack paths exist" is a property of the data, not a fixed display constant. Callers can still pass an explicit `top_n` if a specific view wants the list trimmed.

5.4 · CMDB Grounding (No Pre-Solved Paths)

`backend/core/cmdb.py` parses the enterprise architecture Markdown document via pipe-table regex parsing, auto-detecting between a legacy prose-table format and the newer CI-based, ServiceNow-style format used by both shipped datasets. This part of the parser is materially unchanged — including its known fragility (a prior silent-failure incident where a format mismatch produced an empty CMDB with no error raised anywhere).

What changed: the CMDB no longer carries attack paths at all. The architecture document itself was restructured to add four new grounding sections that the CMDB now parses into condensed, analyst-readable relationship lines:

CMDB FIELD	ARCHITECTURE DOC SECTION	WHAT IT CAPTURES
<code>hosting_relations</code>	§2 Virtualization / Hosting	Which guest CIs run on which host CI — compromise of a hypervisor implies compromise of everything it hosts
<code>reachability_rules</code>	§3 Network Reachability Rule Base	Zone-to-zone firewall rules; rows marked <i>Excessive</i> are misconfigurations that actually open a boundary
<code>dependencies</code>	§4 Host/App/DB Dependencies	Application-tier to database-tier and similar service dependency relationships
<code>cred_relations</code>	§5 Identity & Credential Relationships	A credential or identity valid on more than one CI — the non-network pivot class a zone-only view misses entirely

These four sections plus the zone/asset/team inventory are exactly what `CMDB.grounding_context()` hands to every LLM agent. **The documented attack paths are not in this context at all** — they were

moved entirely out of the ingested data and into the held-out oracle described in Section 6.4. A backward-compatible `summary_text()` alias still exists for any caller expecting the old name, but it now resolves to the same paths-free grounding context, so nothing in the codebase can accidentally leak the answer key to an agent through an old code path.

6 · Persistence, Multi-Tenancy & Run Lifecycle

This is the newest and largest addition to the platform. Previously, "analysis state" meant a single global object in server memory plus one JSON file caching the AI layer's output. That has been replaced with a proper, if intentionally lean, persistence and run-management design.

6.1 · SQLite/SQLAlchemy Persistence

`backend/db/` is a new package with three files:

- `models.py` — one SQLAlchemy model, `AnalysisRun`: id, dataset key, input content-fingerprint, status (`running` / `complete` / `failed`), whether AI was enabled, timestamps, an error message if failed, the four AI-enriched JSON blobs (discovery, correlation, compliance, executive summary), and a small KPI snapshot.
- `repository.py` — the *only* module in the codebase allowed to touch a SQLAlchemy `Session`; every other module deals in `AnalysisRun` instances or plain summary dicts. Provides `create_run`, `complete_run`, `fail_run`, `get_run`, `latest_complete_run`, `list_runs`, `delete_run`, `delete_all_runs`.
- `session.py` — engine + sessionmaker, resolved from `DATABASE_URL` (defaults to a local SQLite file, `sqlite:///backend/data/voc.db`); `init_db()` is called once at FastAPI app startup (`main.py`) and creates the schema if it doesn't exist — no manual migration step for the SQLite default.

Deliberately lean by design (stated directly in the code's own docstring): the deterministic layer — GRS scoring, capability classification, the reachability graph — is a pure, fast function of the input files, so it is *never* stored, only recomputed on load. What *is* persisted is exactly what's expensive to regenerate: the LLM-enriched discovery/correlation/compliance/executive-summary blob, plus a lightweight KPI snapshot captured on every run (complete *or* deterministic-only) so trend/delta reporting has data to work with even for an operator with no AI provider connected.

Swapping SQLite for Postgres in a production deployment is a single `DATABASE_URL` environment variable change — nothing else in the application code changes, since every access already goes through `db/repository.py`'s SQLAlchemy layer.

6.2 · Runtime-Switchable Enterprises

`backend/core/datasets.py` is a new dataset registry that replaces what used to be a hardcoded `VULN_CSV_PATH` / `ARCHITECTURE_MD_PATH` pair in `config.py`. It works by filesystem convention:

An "enterprise" is scannable when the data directory contains **both** a `<name>_vulnerabilities.csv` and a `<name>_architecture.md`. `discover()` re-scans the filesystem on every call, so dropping a third matching pair into `backend/data/` makes it selectable immediately, with no restart and no code change.

For each discovered enterprise, a persona (display name, sector, compliance frameworks) is resolved by: (1) a built-in override for the two shipped labs (Ghrab Financial Group, Velon Health Systems); else (2) auto-parsing the architecture document's own H1 heading plus optional `Sector:` / `Frameworks:` lines

near the top; else (3) a generic fallback persona (ISO 27001 / NIST CSF framing). This persona feeds directly into every agent's system prompt (Section 7.3) so narratives describe the right sector and regulatory context automatically.

Switching the active enterprise (`POST /api/datasets/select`) flushes the cached vulnerabilities DataFrame and parsed CMDB, drops the in-memory analysis, and the SPA re-fetches everything against the new dataset — see the **DatasetPicker** component in Section 9. On a cold boot the platform defaults to Velon if present, else the first dataset discovered.

6.3 • Duplicate Detection, Reuse & the Kill Switch

Three new operator-facing behaviors came with the persistence layer, all implemented cooperatively between `api/state.py` and `api/streams.py`:

CONTENT FINGERPRINTING & DUPLICATE DETECTION

`core/fingerprint.py` computes a SHA-256 hash over the active dataset's vulnerability CSV, its architecture Markdown, and the shared risk-methodology document — deliberately *not* over runtime knobs like which LLM provider is assigned to which agent, since a provider swap is a deliberate operator choice that should trigger a fresh run rather than a silent duplicate match. Before starting a new pipeline run, the platform checks whether this exact fingerprint already has a completed run on file; if so, the SSE stream emits a `duplicate` event instead of silently re-spending LLM calls, and the SPA surfaces a card offering **Reuse results** (instant, zero agent calls — `POST /api/analyze/reuse/{run_id}`) or **Refresh anyway** (forces a new run).

RUN HISTORY

`GET /api/analyze/runs` lists every persisted run for the active dataset (lightweight — KPI snapshots only, not the full AI blobs); individual runs or the whole history can be deleted. If the deleted run was the currently active one, the in-memory analysis is dropped so the next request cold-hydrates from whatever's now the latest surviving run.

OPERATOR KILL SWITCH

A running pipeline can now be cancelled mid-flight. Cancellation is **cooperative**, not forceful:

`run_ai_layer()` polls a `should_cancel` callback at every agent boundary (before Analyst Detection, before Discovery, before Correlation, before Compliance, before Triage) and raises `RunCancelled` the next time it's checked — a kill takes effect between stages rather than tearing down a provider call mid-request. The cancelled run is recorded in the database with status `failed` and an explanatory error, and never becomes the active analysis. The SPA exposes this as a red "Kill switch" button that replaces the Full Analysis/Re-run controls while a pipeline is running.

6.4 • The Held-Out Verification Oracle

This is the most significant conceptual change in the current design. Previously, the "documented" attack paths (PATH-A through PATH-F) that the engine is graded against lived inside the same ingested CSV, in an `Attack_Path_Ref` column — technically unused by the detection logic, but physically present in the data the platform loads. That has been removed entirely.

The answer key now lives in `backend/data/oracle/` , structurally isolated from ingestion. Two files per enterprise: a CSV (`<name>_attack_paths.csv` , QID → documented path/step reference, e.g. `PATH-E-Step3`) and a human-readable Markdown narrative with the fully cross-referenced chains, each step citing its exact enabling Rule ID, Relationship ID, or Credential Relationship ID.

`core/oracle.py::attack_path_refs()` is the only reader of this directory, and it is loaded by exactly two callers in the entire codebase: `core/graph.py::build_chains()` (which reconstructs the documented chains purely for the "discovered vs. ground-truth" verification overlay) and `tests/test_rediscovery.py` (the acceptance gate). **No agent, and no part of the detection pipeline, ever imports `core.oracle` .**

This structural separation is what makes the platform's strongest claim possible: that its detection is genuinely independent, not merely "doesn't read a column it could have read." The `Attack_Path_Ref` data physically does not exist anywhere the engine or an agent can reach it.

THE VERIFICATION ENDPOINT

A new API surface, `GET /api/verification` , grades two independent detection signals against this held-out oracle simultaneously:

- **Engine rediscovery** — does the deterministic reachability graph independently connect INTERNET → the documented entry → the documented crown-jewel target?
- **AI detection** — did the Analyst Detection Agent (Section 7.3) independently name that same target as a path terminus, reasoning from raw grounding alone?

Both scores are surfaced together in the Attack Paths screen (Section 9) as an "N/6 rediscovered by the reachability engine" / "N/6 detected by the analyst agent" pair — giving an operator (or this VM team) two separately falsifiable signals that the system's detection claims are real, rather than one score that could be gamed by either layer quietly leaning on the other.

7 • The AI Agent Layer

7.1 • Orchestrator & Pipeline

`agents/orchestrator.py` is the coordination point — not itself an AI agent. As described in Section 4, it is split into `compute_deterministic()` (cheap, always safe to call) and `run_ai_layer()` (token-spending, invoked only from the explicit analyze action). The AI stage order is now:



A full run costs roughly 6–7 LLM calls total. The Analyst Detection Agent runs *first* deliberately, before it can be influenced by anything the deterministic engine or the other agents concluded. The Triage Agent still runs last and is fed a compact digest of what the Discovery, Correlation, and Compliance agents already concluded, so the dashboard's headline executive summary stays consistent with the detail panels beneath it. The Remediation Agent remains excluded from this bulk pipeline, invoked only on-demand per finding — its results are now cached per-QID on the in-memory `AnalysisResult` (`remediations: dict[qid, result]`) and retrievable via a new `GET /api/findings/{qid}/remediation` so a page refresh doesn't lose a previously generated remediation plan.

7.2 • Multi-Provider Architecture (litellm)

Unchanged in design from the prior revision: no agent calls a vendor SDK directly — every call goes through `core/providers.py::call_llm()`, built on litellm's unified interface. Five agent *roles* (not seven — two of the seven agent modules share the `attack_path` role, see Section 7.3) each have a preferred provider plus an automatic fallback chain:

ROLE	DEFAULT PROVIDER	DEFAULT MODEL	USED BY
attack_path	Mistral	mistral-large-latest	Discovery Agent (narration), Attack Path Agent, Analyst Detection Agent
correlation	Groq	llama-3.3-70b-versatile	Discovery Agent (toxic combos), Correlation Agent
compliance	Mistral	mistral-large-latest	Compliance Agent
triage	Mistral	mistral-large-latest	Triage Agent (chat + executive synthesis)
remediation	Groq	llama-3.3-70b-versatile	Remediation Agent

Fallback order if the preferred provider has no key or fails: `mistral → groq → gemini → anthropic → openai → deepseek → xai`. Client-side rate limiting is unchanged: consecutive calls to the same provider are spaced by a tuned minimum interval (Groq 2.5s, Gemini 4.5s, Mistral 1.5s, others 4.0s default), and a 429 triggers bounded same-provider retry-with-backoff before falling through the chain — all tunable via environment variables.

The shared analyst persona (`agents/base.py::analyst_persona()`) is now built fresh on every call rather than as a fixed module constant, so it always frames the *currently active* enterprise's name, sector, and compliance frameworks (via `core/config.py::active_org()`) — a direct consequence of multi-tenancy: an agent narrating findings for Velon Health Systems must not describe itself as serving Ghrab Financial Group. The persona text itself was also rewritten to make explicit that attack paths are *not* handed to the model — it must derive them by chaining CMDB relationships, and every hop it asserts must trace to a specific relationship and finding in context.

The Agent Activity Log (`core/call_log.py`) is unchanged: an in-memory, pub-sub log of every LLM call attempt (start/success/error, provider, model, duration, tokens, best-effort USD cost), feeding the live "Agent Logs" screen. Still explicitly not persisted to the database — cleared on backend restart.

7.3 · The Seven Agents, In Detail

① Analyst Detection Agent

NEW

`agents/analyst_agent.py::detect_attack_paths()` — the platform's newest and most ambitious agent. Every other agent in this system is handed some form of pre-computed structure to validate or narrate. This one is handed **nothing but raw grounding**: the asset inventory, ownership, zone reachability rules, credential-reuse and dependency relationships, plus a table of what each finding grants an attacker (from the deterministic capability classifier) — and separately, which hosts are internet-reachable entry points and which are crown jewels. No candidate chains, no documented answer key, nothing resembling a path is in its context.

From that alone it is asked to trace the 4–6 most consequential end-to-end attack paths, citing for every hop the enabling QID *and* the specific CMDB relationship (a Rule ID, a credential relationship, or a dependency) that makes the hop real — "a hop with no enabler is not a real hop." Every proposed hop is then independently verified: hosts are resolved against the real asset set, QIDs against the real findings set; a fabricated QID is stripped rather than trusted, and a path is marked `grounded` only when every one of its hops checks out. This is what lets the platform claim its *agents* detect attack paths, not just its rule engine — and it gives the verification view (Section 6.4) a second, fully independent detection signal.

② Discovery Agent

`agents/discovery_agent.py::analyze()` — the reasoning layer directly on top of the deterministic path engine. Distinct from the Analyst Detection Agent above: this one is handed the graph engine's candidate chains and validates/ranks/narrates them, in two bounded passes: (1) per-chain headline, hop-by-hop narrative, business impact, single choke-point, confidence (high/medium/low), and novelty (textbook/notable/non-obvious); (2) reasoning *beyond* the enumerated set for up to 4 cross-path toxic combinations (shared credentials, flat trusts merging otherwise-separate chains) plus an executive attack-surface synthesis. Never invents hosts or QIDs — the candidates are ground truth handed to it.

③ Correlation Agent

`agents/correlation_agent.py::find_toxic_combinations()` — cross-references the full findings table against the CMDB's reachability/credential/dependency grounding (not any pre-solved path) to surface risk an operator scanning top-to-bottom would miss: correlated findings whose combined mechanism it can name from a real CMDB relationship, the top risk-owning teams by aggregate exposure, and mis-prioritized findings whose true blast radius exceeds their apparent severity.

④ Compliance Agent

`agents/compliance_agent.py::compliance_summary()` — reasons over the `Compliance_Ref` column plus the CMDB's regulated-scope facts (PCI CDE, SWIFT CSP scope, DORA critical-function status) into a posture briefing: frameworks in scope, key gaps grounded to specific QIDs, a note on what the DORA CIF SLA overlay means operationally, and an auditor/board-readable executive summary. Explicitly framed to flag gaps rather than assert certification status the platform has no authority to know.

⑤ Triage / Analyst Agent

`agents/triage_agent.py` — powers the free-form AI Analyst chat (grounded Q&A over CMDB + findings + discovered paths, with conversation history) and the one-paragraph executive synthesis on the Command Center. Runs last in the pipeline, fed a digest of what the three prior agents already concluded and instructed to stay consistent with that prior work rather than re-deriving an independent, possibly contradictory view.

⑥ Remediation Agent

`agents/remediation_agent.py::enrich_remediation()` — on-demand only, triggered when an operator drills into a single finding. Enriches the scanner's raw remediation text into a plain-English risk summary, an ordered step-by-step procedure, validation steps, a note on any operational risk the fix itself introduces, and an effort estimate. Now includes output-shape normalization (`_coerce_lists` / `_coerce_strings`): since the model is only told via prose instruction that a field is "an ordered list" or "a short string," nothing structurally enforces that shape, and a model can return a single string where a list was expected (or a nested object where a string was expected) — this would previously reach the frontend as either a broken `.map()` call or an uncaught React render crash on trying to render an object as a child. Both are now flattened server-side before the response leaves the API.

⑦ Attack Path Agent

`agents/attack_path_agent.py::narrate_chain()` — narrates a single, already-built deterministic chain (from the oracle-derived `Chain` structure) into headline / narrative / business impact / choke point / owning teams. Used for the documented-path overlay rather than the bulk discovered-path list; the step sequence is ground truth and never invented.

7.4 • Anti-Hallucination / Grounding Design

Every JSON-mode agent output is post-parse validated against deterministic ground truth before it reaches the API/UI, at zero extra LLM-call cost — unchanged in principle from the prior design, now extended to cover the new agent:

- **Analyst Detection Agent:** every hop's `from / to` host is resolved against the real asset set (best-effort fuzzy match); a fabricated QID is stripped from the hop rather than trusted; a path with no anchorable hop, or whose target isn't a real asset, is dropped entirely; a path is labeled `grounded` only when every hop verifies.
- **Discovery Agent:** a narrated `path_id` absent from the graph engine's actual candidate set is dropped and reported as `hallucinated_path_ids`; a `choke_point` whose text doesn't cite a real QID is flagged `choke_point_unverified`; toxic combinations citing zero real QIDs are dropped entirely.
- **Correlation Agent:** reprioritization flags citing an unknown QID are dropped, with a `dropped_reprioritization_flags` count surfaced.
- **Compliance Agent:** gaps citing zero real QIDs are dropped entirely — an ungrounded compliance claim is worse than none.

8 • API Surface (REST & SSE)

All endpoints are prefixed `/api`. Full interactive documentation is auto-generated by FastAPI at `/docs`. 29 endpoints total, up from 18 in the prior revision — the growth is almost entirely the new dataset and run-lifecycle surface.

METHOD & PATH	PURPOSE
GET <code>/health</code>	Liveness check
GET <code>/datasets</code> NEW	Every scannable enterprise found in the data directory, plus which is active
POST <code>/datasets/select</code> NEW	Switch the active enterprise; flushes cached CSV/CMDB + in-memory analysis
GET <code>/overview</code>	Command Center payload: KPIs, executive summary, CVSS-vs-GRS scatter data, top 5 findings, top 5 attack paths, action-band table
GET <code>/kpis</code>	Just the KPI tile numbers
GET <code>/findings</code>	Full findings list, GRS-sorted, now carrying <code>discovered_path_refs</code> instead of the removed <code>attack_path_ref</code>
GET <code>/findings/{qid}</code>	Single finding detail incl. GRS factor breakdown
GET <code>/findings/{qid}/remediation</code> NEW	Previously generated remediation plan for this QID, if any (session-cached)
POST <code>/findings/{qid}/remediation</code>	Triggers the Remediation Agent for one finding (requires AI provider — 409 otherwise)
GET <code>/attack-paths</code>	Discovered + narrated paths, <code>ai_detected</code> paths NEW , toxic combinations, documented-path overlay
GET <code>/verification</code> NEW	Grades engine + AI-agent rediscovery of the held-out documented paths
GET <code>/graph</code>	Full node/edge payload for the Cytoscape graph
GET <code>/correlation</code>	Correlation Agent output
GET <code>/compliance</code>	Compliance Agent output
GET <code>/teams</code>	Per-team exposure rollup
GET <code>/settings/providers</code>	Provider configuration state + current role → provider assignments
POST <code>/settings/providers</code>	Set/clear an API key for a provider (invalidates the cached analysis)
POST <code>/settings/agent</code>	Override which provider serves a given agent role
GET <code>/analyze/status</code>	Lets a freshly loaded page discover an already-in-flight run
POST <code>/analyze</code> SSE	Kicks off (or attaches to) the pipeline; now also emits <code>duplicate</code> and <code>cancelled</code> events

POST /analyze/cancel NEW	Operator kill switch — signals the in-flight run to stop at its next agent boundary
POST /analyze/reuse/{run_id} NEW	Adopts a previously persisted run as active without touching the agent chain
GET /analyze/runs NEW	Run history for the active dataset (KPI-snapshot level)
DELETE /analyze/runs/{run_id} NEW	Delete one persisted run
DELETE /analyze/runs NEW	Wipe all persisted history for the active dataset
POST /chat SSE	AI Analyst chat — streams the Triage Agent's answer
GET /logs · DELETE /logs	Full LLM call history / clear it
GET /logs/stream SSE	Live tail of every LLM call, with an authoritative "currently active" resync snapshot on each (re)connect

9 • Frontend Application

Still a single-page "command-center" UI on 9 routed screens under a shared sidebar/header layout — the route list itself is unchanged, but the layout shell and several screens gained substantial new capability:

SCREEN	ROUTE	WHAT IT SHOWS
Command Center	/	KPI tiles (now count-up animated NEW), executive synthesis, band-distribution + CVSS-vs-GRS charts, most-urgent findings, top attack paths
Findings	/findings	Sortable/filterable/searchable table; click-to-drill slide-over drawer; path membership now shown via engine-derived <code>discovered_path_refs</code>
Attack Paths	/attack-paths	Interactive attack-surface graph + ranked path rail, verification scorecard NEW , Analyst-Detected Paths panel NEW , toxic combinations
Correlation	/correlation	Cross-finding insights, top risk-owning teams, reprioritization flags
Teams	/teams	Per-team peak-risk bar chart + exposure cards, click-through to filtered findings
Compliance	/compliance	Frameworks in scope, DORA CIF overlay note, key gaps grounded to QIDs
AI Analyst	/analyst	Streaming chat grounded in live findings/CMDB/attack-path data
Agent Logs	/logs	Live + historical feed of every LLM call, filterable, exportable to CSV
Settings	/settings	Per-provider API key entry, per-role provider assignment

9.1 • New Application-Shell Capabilities

The biggest visible change is in `components/Layout.tsx` and three new standalone components:

- **Enterprise switcher** (`components/DatasetPicker.tsx`) — a header button + modal listing every enterprise the backend discovered, each showing its finding count and compliance frameworks; selecting one calls `POST /datasets/select`, invalidates every cached query, and toasts a confirmation.
- **Command palette** (`components/CommandPalette.tsx`) — a ⌘K/Ctrl+K-triggered, keyboard-navigable action launcher covering every nav route plus operator actions (switch enterprise, run full analysis, re-run, export report, toggle theme), grouped by section with fuzzy label/keyword matching.
- **Toast notification system** (`lib/toast.tsx`) — an app-wide provider rendering a top-right stack (positioned to never collide with the AI dock or the run-progress overlay), auto-dismissing after ~4.2s, used throughout the app for confirmations (enterprise switched, analysis complete, export succeeded) and failures.
- **Kill switch UI** — while a pipeline is running, the header's Full Analysis/Re-run buttons are replaced by a red "Kill switch" control (also present, smaller, inside the progress overlay itself) that calls `POST /analyze/cancel`; the progress overlay now distinguishes complete / error / **cancelled** states.
- **Duplicate-run card** — when the backend reports the input is byte-identical to an already-completed run, a bottom-center card offers **Reuse results** or **Refresh anyway** instead of silently spending

tokens.

- **Animated route transitions** — framer-motion cross-fades the outlet content on every navigation; the whole app also now scrolls to top on route change.
- **Count-up KPI tiles** (`lib/useCountUp.ts`) — Command Center numbers ease in from their previous value rather than snapping, honoring the OS reduce-motion setting.
- **Generic error boundary** (`components/ErrorBoundary.tsx`) — a reusable version of the graph-specific error boundary from the previous revision, now available to wrap any subtree so a rendering error doesn't blank the whole app.

9.2 · Verification Scorecard & Analyst-Detected Paths

The Attack Paths screen gained two new panels directly surfacing the held-out oracle design from Section 6.4:

- A scorecard reading " `N/6 rediscovered by the reachability engine` " and " `N/6 detected by the analyst agent` " side by side, with the explanatory note that documented paths are never ingested and never shown to the engine or the agents.
- An "Analyst-Detected Paths" section rendering the Analyst Detection Agent's independently-reasoned chains — each hop shown with a colored dot (verified vs. not), its citing QID and CMDB enabler, and a `grounded` / `N/M verified` badge per path.

9.3 · Attack Graph Visualization

Unchanged from the prior revision — still a purpose-built Cytoscape.js canvas renderer (`features/attackpaths/AttackGraph.tsx`) with three switchable layouts (Flow / Zones / Radial), hop-by-hop path replay with play/pause/scrub, a docked node inspector, edge-kind filtering, a minimap, fullscreen, and full light/dark theme support via a canvas stylesheet rebuilt from resolved CSS custom properties.

10 · Deployment & Operations

Local development

Backend	<code>cd backend && uvicorn main:app --reload --port 8000</code>
Frontend	<code>cd frontend && npm run dev</code>
Both at once	<code>./dev.sh</code>

On first startup the backend creates `backend/data/voc.db` automatically (SQLite, zero-ops) — nothing to install or migrate by hand. Point `DATABASE_URL` at a Postgres instance for multi-instance/production deployment; the application code doesn't change either way.

Docker

`docker compose up --build` starts two services:

- **backend** — Python 3.12-slim, installs `requirements.txt` (now including `sqlalchemy>=2.0`), runs uvicorn on `:8000`. **The bind-mounted volume now covers the entire `backend/data` directory** (previously only the cache subfolder), so the SQLite database, the analysis cache, and the oracle files all survive container restarts.
- **frontend** — multi-stage build (Node 22 → static Vite bundle served by nginx 1.27-alpine); nginx redirects HTTP → HTTPS, terminates TLS, serves the SPA with client-side-routing fallback, and reverse-proxies `/api/` to the backend with buffering disabled so SSE frames flush immediately.

Provider API keys are supplied via a root `.env` file (see `.env.example` — all seven keys are optional; the platform runs fully deterministic with zero configured).

No CI/CD pipeline exists. There is no `.github/workflows` directory, no automated build/test/lint gate on push or PR, and no automated Docker image publishing. Every deploy today is a manual `docker compose up --build` on the target host. This is unchanged from the prior revision and, if anything, more consequential now that there's a database schema to keep in sync across environments.

11 · Testing & Quality Assurance — Current State

Still exactly one automated test in the entire codebase: `backend/tests/test_rediscovery.py`, updated to read from `core.oracle.attack_path_refs()` instead of a CSV column, but functionally the same acceptance gate as before — it runs the full deterministic pipeline and checks that the engine's own reachability graph independently proves a path from INTERNET to each documented attack path's entry and target host, using the held-out oracle strictly as an offline grading reference it never feeds into the engine.

Beyond that single test, there is still no automated test coverage anywhere in the platform — and the persistence/multi-tenancy layer added this cycle is entirely untested:

- No unit tests for the GRS formula, capability classification, or the new content-fingerprint/duplicate-detection logic.
- No tests for the new SQLAlchemy models or repository CRUD functions — a schema change or a bad migration would currently be caught only by manual testing.
- No tests for the dataset auto-discovery/switching logic, including the persona-derivation regexes that parse a new `architecture.md`'s header.
- No API/integration tests for any of the 29 REST/SSE endpoints.
- No frontend tests of any kind — zero `*.test.*` / `*.spec.*` files exist in `frontend/src`, including for the new CommandPalette, DatasetPicker, kill-switch, and duplicate-detection UI flows.
- No CI workflow runs even the one existing test automatically on push/PR.

12 · Recommended Improvements to the Tech Stack

Gaps identified directly from reading the current implementation, ranked by how much they matter before this platform could responsibly hold real organizational vulnerability data or be exposed beyond a single trusted operator's laptop. Several items from the prior revision were partially or fully closed by this cycle's work (noted inline); new gaps opened up by the persistence/multi-tenancy layer are marked accordingly.

Security & multi-user readiness

No authentication or authorization anywhere in the API

HIGH

Still true, and now higher-stakes: every endpoint — including the new dataset-switching and run-deletion endpoints, and the provider-key endpoint — is open to any client that can reach the backend. Any operator on the network can wipe another operator's run history (`DELETE /analyze/runs`) or switch the active enterprise out from under a concurrent session. This needs at minimum an auth middleware (a bearer-token gate) before any deployment beyond a single trusted operator's machine.

Secrets still held in plaintext, in-memory, process-wide

HIGH

`api/state.py::runtime_settings` remains a single process-wide dict — unchanged from before. No encryption at rest, no secrets-manager integration, and now shared across however many enterprises the operator has registered.

No isolation between concurrently-connected operators viewing different enterprises

HIGH

NEW
GAP

The multi-tenancy added this cycle is *switchable*, not *concurrent* — there is exactly one active dataset and one active in-memory analysis per backend process. If two operators are using the same backend instance and one switches the enterprise, the other's view silently changes underneath them (their next query returns different-org data with no warning). A real multi-tenant deployment needs the active dataset to be a request-scoped or session-scoped value, not a global.

No rate limiting on cost-incurring endpoints

MEDIUM

Unchanged: `POST /analyze` and `POST /chat` both trigger real LLM spend with no platform-level cost guardrail. The new duplicate-detection logic (Section 6.3) helps avoid *redundant* spend on identical input, but does nothing to stop a client from calling `/analyze?force_refresh=true` in a loop.

Data & persistence

Deterministic layer trend history still doesn't exist, despite the new database

MEDIUM

PARTIALLY
CLOSED

The prior revision's biggest persistence gap — no way to see risk trending over time — is now half-closed: the `kpi_snapshot` captured on every run *does* give band-distribution/avg-GRS history per dataset, and `GET /analyze/runs` exposes it. But nothing in the frontend currently visualizes this run history as a trend — there is no chart plotting KPI snapshots over time, so the data is being captured but not yet surfaced. This is now a frontend gap, not a backend one.

EPSS and KEV data are still hardcoded lookup tables, not live feeds

HIGH

Unchanged: `risk_engine.py`'s `EPSS_BY_CVE` and `KEV_CVES` cover only the CVEs used across the shipped lab datasets. This matters more now that the platform actively invites additional enterprises to be dropped in (Section 6.2) — a third dataset with unrecognized CVEs will silently score `EPSS=0.0` for every one of them, zeroing out 15% of GRS's weight with no warning surfaced anywhere. This must become a live FIRST.org/CISA KEV integration before the multi-tenancy feature is used with anything beyond the two curated labs.

CMDB ingestion is still regex-parsed Markdown

MEDIUM

Unchanged fragility (Section 5.4) — now parsing a larger document with four additional relationship sections, which is more surface area for the same class of silent-failure risk that already bit this platform once. A structured input format (JSON/YAML CMDB export, or a real CMDB system-of-record integration) remains the durable fix.

Dataset auto-discovery has no validation or sanitization

LOW

NEW GAP

`core/datasets.py::discover()` scans the data directory by filename convention and derives an org persona by regex-parsing whatever Markdown is dropped in, with no schema validation of the CSV columns or the architecture doc's section structure before it's presented as selectable in the UI. A malformed or incomplete dataset pair will surface as a selectable "enterprise" that then fails confusingly (empty CMDB, missing columns) only once selected and analyzed, rather than being caught at discovery time.

No live scanner integration

MEDIUM

Unchanged: the platform ingests a static CSV snapshot per enterprise. This is now more natural to close given the persistence layer — a scheduled ingestion job could create new dated `analysis_run` rows automatically, which is exactly what the run-history feature above is already positioned to visualize.

AI / LLM layer

Correlation and Remediation still default to the weakest model for hard reasoning tasks

MEDIUM

Unchanged from the prior revision. Worth revisiting now that the new Analyst Detection Agent — arguably the hardest reasoning task in the whole pipeline, since it gets zero pre-computed structure — defaults to Mistral Large via the shared `attack_path` role, while Correlation (a comparably hard cross-referencing task) still defaults to Groq's fastest/cheapest model.

Chat "streaming" is still simulated, not real token streaming

LOW

Unchanged: `api/streams.py::chat()` waits for the full response, then re-emits it word-by-word with a fixed delay.

Agent call history is still not persisted

MEDIUM

NOW INCONSISTENT

This is a sharper gap than before: the platform now persists AI-enriched *results* to a database but still doesn't persist the AI *call history* that produced them (`core/call_log.py` remains in-memory-only, wiped on restart). A run row in the database can no longer be traced back to which provider/model actually served it, or whether it fell back mid-run — an inconsistency worth closing given how much persistence infrastructure now exists right next to it.

Testing, CI/CD & observability

Near-zero automated test coverage, now including an untested database layer

HIGH

As detailed in Section 11 — this is the single highest-leverage investment before further feature work, and the new persistence/dataset-switching code is the least-tested part of the entire platform despite being its largest recent addition.

No CI/CD pipeline

MEDIUM

Unchanged. More consequential now that a database schema exists to keep in sync across environments with no migration tooling (SQLAlchemy's `create_all` only creates missing tables — it does not alter existing ones, so a future model field addition needs a real migration story, e.g. Alembic, before this bites in production).

No production observability

MEDIUM

Unchanged: no structured logging, error tracking, or metrics/tracing anywhere in the stack.

Frontend

No route-level code splitting

LOW

Unchanged: `App.tsx` statically imports all 9 feature screens; the bundle is now larger still with the addition of the command palette, dataset picker, and toast system to every route's initial load.

Run history is captured but not visualized

LOW

NEW GAP

As noted above — the KPI-snapshot trend data now exists server-side with no corresponding frontend view. This is likely the most natural "next feature" given how much of the backend groundwork is already in place.

13 • Glossary

TERM	MEANING
GRS	Ghrab Risk Score — this platform's composite, deterministic risk score (Section 5.1)
CVSS	Common Vulnerability Scoring System — the industry-standard technical-severity score, one input to GRS
EPSS	Exploit Prediction Scoring System (FIRST.org) — probability a CVE is exploited in the next 30 days
KEV	CISA's Known Exploited Vulnerabilities catalog — binary "is this being exploited in the wild right now" gate
ACW	Asset Criticality Weight — how much a given asset matters to the business/regulatory scope
TCM	Toxic Combination Score — how much worse a finding is because of what it connects to
CCF	Compensating Controls Factor — discount for verified mitigating controls (currently unused, always 1.0)
DORA / CIF	EU Digital Operational Resilience Act; "Critical or Important Function" — a regulated-scope flag driving the 30-day SLA overlay
Crown jewel	A host the discovery engine has identified as a high-value attack target
Toxic combination	A cross-finding risk pattern not visible from any single finding in isolation
Enterprise / dataset	A scannable organization — one <code>*_vulnerabilities.csv</code> + <code>*_architecture.md</code> pair in the data directory, switchable at runtime
Analysis run	One persisted execution of the pipeline against a dataset — a row in the <code>analysis_run</code> SQLite table
Verification oracle	The held-out, never-ingested documented attack paths (<code>backend/data/oracle/</code>) used solely to grade whether the engine/agents independently rediscover known-good chains
Content fingerprint	A SHA-256 hash over a dataset's input files, used to detect byte-identical re-analysis and offer result reuse instead of re-spending LLM calls
litellm	The open-source Python library providing one call interface across many LLM vendors — the platform's entire multi-provider abstraction
SSE	Server-Sent Events — the one-way streaming HTTP protocol used for live progress, chat, and log tailing

Prepared by reading the live codebase at `/home/salamnki/Documents/voc` (branch `rework`, commit `0767ee6`) — every claim in this document traces to a specific file read directly. No claim here is inferred from prior documentation; all descriptions of behavior were verified against the current implementation from scratch.